Guia de Implementação — Pokémon Battle

Este documento descreve **o que cada classe faz** e **quais métodos você deve implementar**. Para cada método, você tem a assinatura, o que deve receber, o que deve retornar e a lógica esperada — em nível quase de pseudo-código.

Não é preciso implementar nada além do que está listado aqui.

Estrutura geral

O jogo tem três classes principais:

- **`Pokemon`** — os dados fixos de um Pokémon (nome, tipos, atributos, ataques que conhece).
- **`Combatente`** — um Pokémon *dentro de uma batalha específica*, com estado próprio (HP atual, ataque sendo carregado).
- **`Batalha`** — orquestra a luta entre dois combatentes.

Além disso, um dicionário global `MULTIPLICADORES` (já pronto) define as interações de tipo entre ataques e defensores.

> **Importante — nomes são um contrato.** Os nomes de atributos e métodos usados neste guia fazem parte do "contrato" da classe: outras partes do código (principalmente a interface do Gradio) esperam encontrá-los exatamente com esses nomes. Siga os nomes indicados à risca — renomear vai quebrar a integração com a interface. Dentro dos métodos, variáveis locais podem ser chamadas do jeito que preferir.

Classe `Pokemon`

Representa um Pokémon do jogo: seus atributos base e a lista de ataques que ele conhece. Um `Pokemon` é apenas dado — ele não luta sozinho.

`__init__(self, nome, tipos, atk, dfn, hp, spd)`

Construtor da classe.

Parâmetros:

- `nome` (str): nome do Pokémon.
- `tipos` (list[str]): lista com um ou dois tipos, ex.: `["fogo"]` ou `["grama", "veneno"]`.
- `atk` (int): atributo de ataque.
- `dfn` (int): atributo de defesa.
- `hp` (int): pontos de vida máximos.

- `spd` (int): velocidade (define quem ataca primeiro).

O que fazer:

1. Guardar cada parâmetro como atributo do objeto, com o mesmo nome:
    - `self.nome = nome`
    - `self.tipos = tipos`
    - `self.atk = atk`
    - `self.dfn = dfn`
    - `self.hp = hp`
    - `self.spd = spd`
2. Iniciar `self.ataques_especiais` como uma lista vazia.
3. Iniciar `self.imagem_frente = None` e `self.imagem_costas = None`.

Os atributos `imagem_frente` e `imagem_costas` **não devem ser alterados ou implementados** — são preenchidos por outra parte do código.

`_calcular_carregamento(self, poder)`

Método auxiliar (uso interno, por isso o `_` no nome). Dado o poder de um ataque, decide quantos turnos ele leva para carregar antes de ser executado.

Parâmetros:

- `poder` (int): poder bruto do ataque.

Retorno: int (número de turnos de carregamento).

Lógica:

- Se `poder >= 800` → retorna `3`
- Senão, se `poder >= 400` → retorna `2`
- Senão, se `poder >= 250` → retorna `1`
- Senão → retorna `0` (ataque instantâneo)

Atenção: a ordem dos `if` / `elif` importa. Comece pelo valor mais alto. Se começar pelo menor, todos os poderes altos vão entrar no primeiro `if` e o resto nunca será alcançado.

`adicionar_ataque(self, nome, poder, tipo)`

Adiciona um novo ataque à lista `self.ataques_especiais` do Pokémon.

Parâmetros:

- `nome` (str): nome do ataque (ex.: `"Chama"`).
- `poder` (int): poder bruto do ataque.
- `tipo` (str): tipo do ataque (ex.: `"fogo"`, `"água"`).

Retorno: nada.

Lógica:

1. Criar um dicionário com as chaves:
    - `"nome"` → `nome`
    - `"poder"` → `poder`
    - `"tipo"` → `tipo`
    - `"carregamento"` → o retorno de `self._calcular_carregamento(poder)`
2. Anexar esse dicionário à lista `self.ataques_especiais`.

`__str__(self)`

Método especial do Python que define como o Pokémon aparece quando você chama `print(pokemon)`. Deve retornar uma string com as informações do Pokémon formatadas em várias linhas.

Parâmetros: nenhum.

Retorno: string.

Lógica:

1. Montar uma string única com os tipos separados por `" / "`. Dica: use `" / ".join(self.tipos)`.
2. Retornar uma string com uma linha para cada informação, no seguinte formato:

```
Nome: {nome}
Tipo: {tipos formatados}
Ataque: {atk}
Defesa: {dfn}
HP: {hp}
Speed: {spd}
Ataques especiais: {ataques_especiais}
```

Cada linha deve ser separada por `\n`. Dica: você pode usar uma única f-string com várias linhas, ou concatenar várias f-strings.

Classe `Combatente`

Representa um Pokémon **dentro de uma batalha**. Guarda o estado que muda durante o combate (HP atual, ataque sendo carregado, turnos restantes) enquanto o `Pokemon` guarda os atributos fixos.

Por que separar `Pokemon` e `Combatente`? Um Pokémon pode entrar em várias batalhas. O HP dele não deveria ficar reduzido para sempre porque ele apanhou uma vez. O `Combatente` funciona como uma "cópia de combate" — quando a batalha acaba, o `Pokemon` original continua intacto.

`__init__(self, pokemon)`

Parâmetros:

- `pokemon` (Pokemon): o Pokémon que este combatente representa.

O que fazer:

1. Guardar o `pokemon` recebido em `self.pokemon`.
2. Iniciar `self.hp_atual` com o valor do HP máximo do Pokémon (`pokemon.hp`).
3. Iniciar `self.ataque_carregando = None` (nenhum ataque sendo carregado no início).
4. Iniciar `self.turnos_restantes = 0` (nenhum turno de carga pendente).

`receber_dano(self, dano)`

Parâmetros:

- `dano` (int): quantidade de dano a ser subtraída do HP atual.

Retorno: nada.

Lógica: subtrair `dano` de `self.hp_atual`.

`esta_vivo(self)`

Verifica se o combatente ainda tem HP.

Parâmetros: nenhum.

Retorno: `True` se `self.hp_atual` for maior que zero, `False` caso contrário.

`esta_carregando(self)`

Verifica se o combatente está no meio de carregar um ataque.

Parâmetros: nenhum.

Retorno: `True` se `self.turnos_restantes` for maior que zero, `False` caso contrário.

Classe `Batalha`

Orquestra a luta entre dois combatentes. Cuida da ordem dos ataques, cálculo de dano, aplicação de efeitos de tipo e do log da batalha.

Nota sobre acesso a atributos do combatente: ao longo desta documentação você verá acessos diretos como `atacante.nome`, `atacante.atk`, `atacante.tipos`, `atacante.esta_vivo` e `atacante.esta_carregando`. Estes acessos assumem uma pequena adaptação no `Combatente` que será apresentada em sala antes da implementação desta classe. Não se preocupe se o seu `Combatente` atual não permite isso — vamos ajustar juntos.

`__init__(self, combatente1, combatente2)`

Parâmetros:

- `combatente1`, `combatente2` (Combatente): os dois participantes da batalha.

O que fazer:

1. Guardar os dois combatentes em `self.combatente1` e `self.combatente2`.
2. Iniciar `self.log = []` (lista vazia que vai acumular mensagens do turno).

`_registrar(self, msg)` — já vem pronto

Adiciona uma mensagem ao log. Use este método sempre que quiser registrar algo que aconteceu na batalha.

Como usar:

```
self._registrar("Charizard usou Chama!")
```

`log_texto(self)` — já vem pronto

Retorna o log inteiro como uma string única, separada por quebras de linha. Usado pelo Gradio para mostrar o log na tela.

`_ordem(self)`

Decide qual combatente age primeiro no turno, com base na velocidade.

Parâmetros: nenhum.

Retorno: uma tupla `(primeiro, segundo)` com os dois combatentes na ordem correta.

Lógica:

- Se `self.combatente1.spd >= self.combatente2.spd`:
  - Retornar `(self.combatente1, self.combatente2)`
- Caso contrário:

- Retornar `(self.combatente2, self.combatente1)`

O combatente com maior `spd` age primeiro. Em caso de empate, `combatente1` vai primeiro.

`_calcular_multiplicador(self, ataque, defensor)`

Calcula o multiplicador de dano baseado nos tipos do defensor.

Parâmetros:

- `ataque` (dict): o ataque sendo usado (com as chaves `"nome"`, `"poder"`, `"tipo"`, `"carregamento"`).
- `defensor` (Combatente): quem vai receber o ataque.

Retorno: float (o multiplicador final).

Lógica:

1. Começar com `multiplicador = 1`.
2. Se o tipo do ataque (`ataque["tipo"]`) **não está** presente como chave em `MULTIPLICADORES`:
   - Retornar `multiplicador` imediatamente (vale 1, sem interação).
3. Caso contrário, percorrer a lista `MULTIPLICADORES[ataque["tipo"]]`. Cada item da lista é uma tupla `(tipo, mult)`:
   - Se `tipo` está presente em `defensor.tipos`, multiplicar `multiplicador *= mult`.
4. Retornar `multiplicador`.

Um Pokémon pode ter dois tipos, então mais de uma multiplicação pode ser aplicada. Ex.: um ataque de água contra um Pokémon fogo/terra pode ativar interações contra os dois tipos.

`_buscar_ataque(self, combatente, nome_ataque)`

Dado o nome de um ataque, encontra o dicionário correspondente na lista de ataques do combatente.

Parâmetros:

- `combatente` (Combatente): dono do ataque.
- `nome_ataque` (str): nome a ser buscado.

Retorno: o dicionário do ataque, ou `None` se não encontrar.

Lógica:

1. Percorrer `combatente.ataques_especiais`.
2. Para cada dicionário `a` da lista, se `a["nome"] == nome_ataque`, retornar `a`.
3. Se o laço terminar sem encontrar, retornar `None`.

`_aplicar_ataque(self, ataque, atacante, defensor)`

Executa um ataque: calcula o dano, aplica no defensor e registra no log.

Parâmetros:

- `ataque` (dict): o ataque sendo executado.
- `atacante`, `defensor` (Combatente).

Retorno: nada.

Lógica:

1. Obter o multiplicador com `mult = self._calcular_multiplicador(ataque, defensor)`.
2. Calcular o dano com a fórmula:

   ```
   dano = poder × multiplicador × atk_atacante ÷ dfn_defensor
   ```

   Traduzindo para Python:

   ```
   dano = ataque["poder"] * mult * atacante.atk / defensor.dfn
   ```

   Converter para inteiro e garantir que o dano mínimo seja 1 (para evitar dano zero por defesa muito alta):

   ```
   dano = max(1, int(dano))
   ```

3. Aplicar o dano no defensor: `defensor.receber_dano(dano)`.
4. Registrar no log, nesta ordem:
   - `f"{atacante.nome} usou {ataque['nome']}!"`
   - `f"Causou {dano} de dano em {defensor.nome}."`
5. Ainda no log, adicionar uma mensagem sobre a eficácia:
   - Se `mult == 0`: `"Não teve efeito..."`
   - Se `mult > 1`: `"Foi super eficaz!"`
   - Se `mult < 1` (mas maior que zero): `"Não foi muito eficaz..."`
   - Se `mult == 1`: não registrar nada (dano normal).

`_agir(self, atacante, defensor, nome_ataque)`

Executa a ação de **um único** combatente no turno. Lida com três casos: combatente morto, combatente carregando um ataque, combatente livre para escolher.

Parâmetros:

- `atacante`, `defensor` (Combatente).
- `nome_ataque` (str): nome do ataque escolhido no dropdown pelo jogador do atacante. Pode ser ignorado se o atacante estiver carregando.

Retorno: nada.

Lógica:

1. **Se o atacante não está vivo** (`not atacante.esta_vivo`), não faz nada. `return` .
2. **Se o atacante está carregando um ataque** (`atacante.esta_carregando`):
    1. Decrementar `atacante.turnos_restantes` em 1.
    2. Se `atacante.turnos_restantes` chegou a zero:
        - Guardar o ataque a ser liberado numa variável: `ataque = atacante.ataque_carregando` .
        - Limpar o campo: `atacante.ataque_carregando = None` .
        - Chamar `self._aplicar_ataque(ataque, atacante, defensor)` .
    3. Caso contrário (ainda faltam turnos), apenas registrar no log:

        ```
        "{atacante.nome} está carregando {atacante.ataque_carregando['nome']} ({atacante.tu
        ```

    4. `return` (encerra a função — o combatente não escolhe novo ataque enquanto carrega).
3. **O atacante está livre para escolher.** Buscar o ataque:
    - `ataque = self._buscar_ataque(atacante, nome_ataque)`
    - Se `ataque` for `None` (nome inválido), registrar no log `f"{atacante.nome} não escolheu um ataque válido."` e `return` .
4. **Se o ataque tem carregamento** (`ataque["carregamento"] > 0`):
    - Guardar em `atacante.ataque_carregando = ataque` .
    - Definir `atacante.turnos_restantes = ataque["carregamento"]` .
    - Registrar no log: `f"{atacante.nome} começou a carregar {ataque['nome']}!"` .
    - **Não aplicar dano ainda** — o dano só vai rolar quando os turnos_restantes chegarem a zero.
5. **Se o ataque é instantâneo** (`ataque["carregamento"] == 0`):
    - Chamar `self._aplicar_ataque(ataque, atacante, defensor)` diretamente.

`executar_turno(self, ataque_c1, ataque_c2)`

O turno completo: ambos os combatentes agem, na ordem determinada pela velocidade.

Parâmetros:

- `ataque_c1` (str): nome do ataque escolhido pelo combatente 1.
- `ataque_c2` (str): nome do ataque escolhido pelo combatente 2.

Retorno: nada.

Lógica:

1. **Limpar o log** do turno anterior: `self.log.clear()`. O log só mostra o turno atual, não o histórico.
2. Descobrir a ordem: `primeiro, segundo = self._ordem()`.
3. Mapear qual combatente escolheu qual ataque. Como você tem os ataques indexados por combatente 1 e 2 (não por primeiro/segundo), monte um dicionário:

```
ataques = {
    self.combatente1: ataque_c1,
    self.combatente2: ataque_c2,
}
```

4. Fazer o primeiro agir: `self._agir(primeiro, segundo, ataques[primeiro])`.
5. Se o segundo ainda está vivo (o primeiro pode ter derrubado ele): `self._agir(segundo, primeiro, ataques[segundo])`.

`vencedor(self)`

Parâmetros: nenhum.

Retorno: o `Pokemon` vencedor, ou `None` se a batalha ainda não acabou.

Lógica:

- Se `self.combatente1` não está vivo → retornar `self.combatente2.pokemon`.
- Se `self.combatente2` não está vivo → retornar `self.combatente1.pokemon`.
- Caso contrário → retornar `None`.

`acabou(self)`

Parâmetros: nenhum.

Retorno: `True` se há um vencedor, `False` caso contrário.

Lógica: chamar `self.vencedor()` e comparar com `None`. Se for diferente de `None`, retornar `True`.

Ordem sugerida de implementação

1. **`Pokemon` inteira** — começa por aqui, é a base de tudo.
2. **`Combatente` inteira** — depende de `Pokemon`, mas é curta.
3. **`Batalha`** — parte mais longa. Sugestão de ordem interna:
    1. `__init__` e `_ordem` (aquecimento).
    2. `_calcular_multiplicador` e `_buscar_ataque` (funções puras, fáceis de testar).
    3. `_aplicar_ataque` (junta cálculo e log).
    4. `_agir` (lida com todos os casos de um combatente).

5. `executar_turno` (orquestra os dois).
6. `vencedor` e `acabou` (encerramento).

Dica: teste cada classe conforme implementa. Crie dois Pokémon, adicione ataques, crie combatentes, inicie uma batalha e chame `executar_turno` manualmente no notebook — assim você valida a lógica antes de plugar tudo no Gradio.